

EXPERIMENT NO: 1

Title

Analysis of Stability and Efficiency of Merge Sort with Duplicate Values

Aim

To analyze the stability and efficiency of Merge Sort by executing it on datasets containing duplicate values, observe the preservation of relative order of equal elements, and compare its performance with Quick Sort.

Objective

- To implement Merge Sort and Quick Sort algorithms.
- To analyze the stability of Merge Sort using duplicate values.
- To compare the execution time of Merge Sort and Quick Sort.
- To study the efficiency and behavior of both sorting algorithms.

Problem Statement

Analyze the stability and efficiency of Merge Sort by executing it on datasets containing duplicate values. Observe how the relative order of equal elements is maintained. Record execution time and compare with Quick Sort. Interpret the results and justify why Merge Sort is considered stable.

Algorithm

Merge Sort Algorithm

1. Divide the array into two equal halves.
2. Recursively sort the left half.
3. Recursively sort the right half.
4. Merge the two sorted halves into a single sorted array.
5. During merging, if two elements are equal, select the element from the left subarray first to preserve stability.

Quick Sort Algorithm

1. Select a pivot element from the array.
2. Partition the array into three parts:
 - Elements smaller than the pivot.
 - Elements equal to the pivot.
 - Elements greater than the pivot.
3. Recursively apply Quick Sort on the left and right partitions.
4. Combine the sorted partitions to obtain the final sorted array.

Source Code

(Insert the Python program here.)

Input Dataset

Original Dataset:

[(5,'A'), (3,'B'), (5,'C'), (2,'D'), (3,'E'), (8,'F'), (5,'G'), (2,'H')]

The dataset contains duplicate values such as 5, 3, and 2.

Output

(Insert output screenshot here.)

Stability Analysis

Original Order of Duplicate Elements:

Value 5 → A, C, G

After Merge Sort:

Value 5 → A, C, G

The order of duplicate elements remains unchanged after sorting. Therefore, Merge Sort preserves the relative ordering of equal elements and is considered a stable sorting algorithm.

Quick Sort does not always guarantee the preservation of the relative order of equal elements and hence is generally considered unstable.

Execution Time Comparison

Algorithm	Execution Time
Merge Sort	_____ seconds
Quick Sort	_____ seconds

(Note: Fill the actual execution times obtained from your program output.)

Time Complexity Analysis

Merge Sort

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n \log n)$

Quick Sort

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n^2)$

Result and Discussion

The program was successfully executed using datasets containing duplicate values. Merge Sort maintained the relative order of duplicate elements, demonstrating its stable nature. Execution time analysis showed that both Merge Sort and Quick Sort perform efficiently for small datasets. However, Merge Sort guarantees $O(n \log n)$ complexity in all cases, whereas Quick Sort may degrade to $O(n^2)$ in the worst case.

Conclusion

Thus, Merge Sort was implemented and analyzed successfully. The results confirmed that Merge Sort is a stable sorting algorithm because it preserves the relative order of equal elements during the merge process. Although it requires additional memory, it provides consistent performance and is suitable for applications where stability is essential.